

Guía de ejecución y despliegue

1. Requisitos previos

Antes de ejecutar el proyecto, tener instalado:

- Git
- Docker Desktop
- Postman

Verificar que Docker Desktop esté iniciado.

2. Clonar el repositorio

Ubicarse en la carpeta donde se desea guardar el proyecto:

```
cd C:\Git
```

Clonar el repositorio:

```
git clone https://github.com/JosethAcostaLoayza/casino-atlantic-city.git
```

Ingresar al repositorio:

```
cd casino-atlantic-city
```

3. Levantar los servicios

Ejecutar:

```
docker compose up -d --build
```

Este comando construye las imágenes y levanta los servicios definidos en docker-compose.yml.

Verificar que los contenedores estén levantados:

```
docker compose ps
```

Todos los servicios necesarios deberían encontrarse en estado Up o Running.

4. Probar los servicios con Postman

Abrir Postman e importar la colección ubicada en:

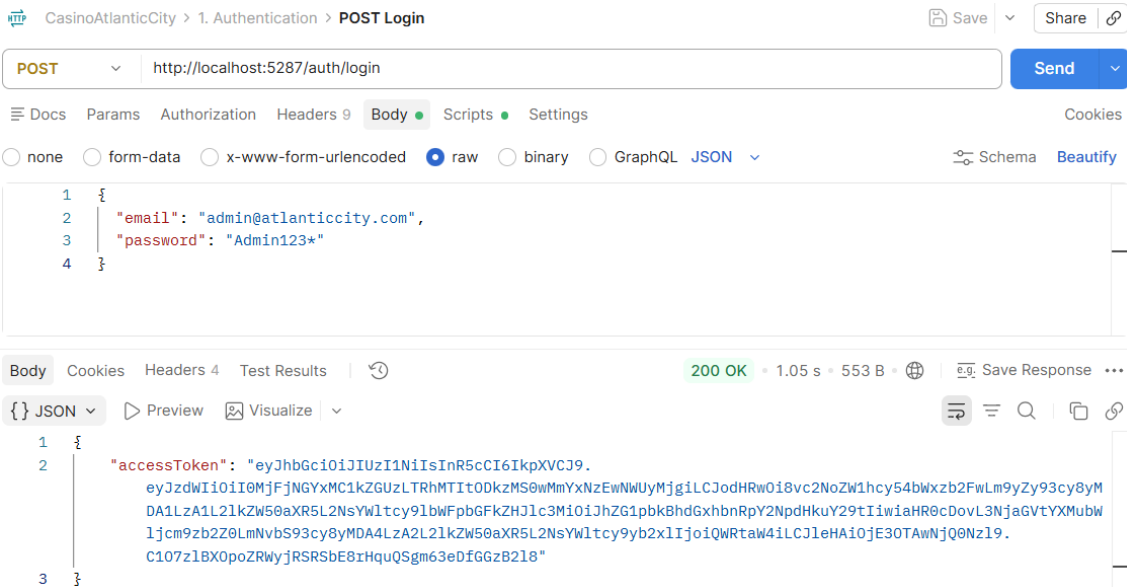
/postman/AtlanticCity.postman_collection.json

4.1 Login

Ejecutar: POST /login

Utilizar las credenciales indicadas en la colección de Postman.

El login devolverá el token de autenticación necesario para consumir los endpoints protegidos.

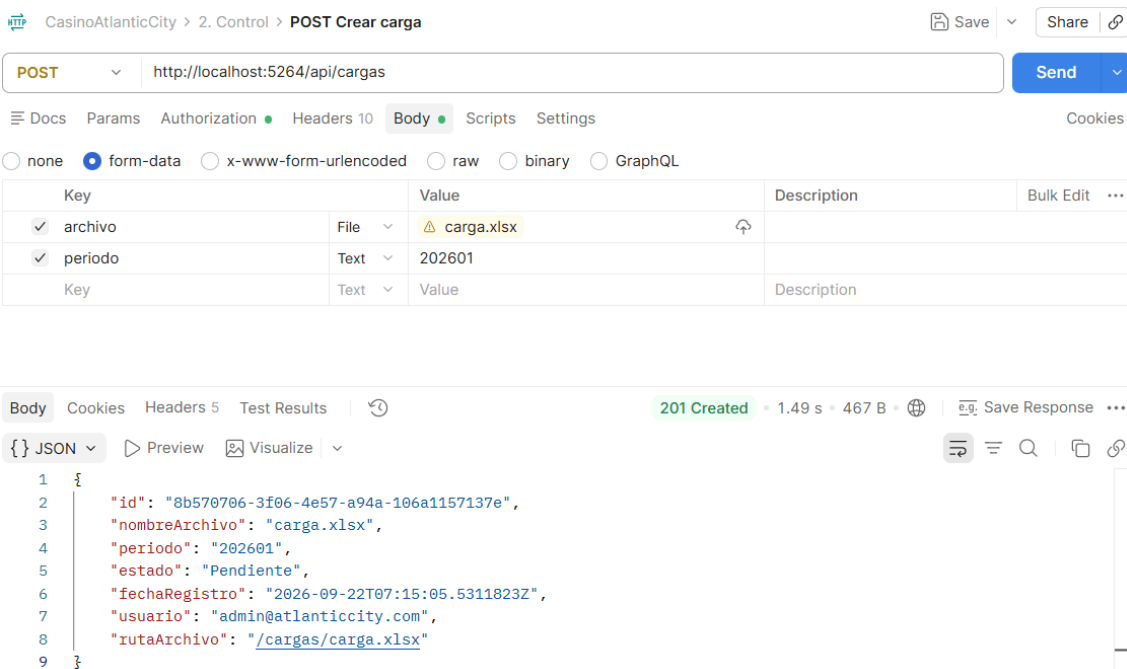


4.2 Carga masiva (docker compose logs -f carga-masiva)

Luego de autenticarse, ejecutar el endpoint de carga masiva:

POST /cargas

Adjuntar el archivo Excel de prueba.



4.3 Consultar contenido de archivo Excel

CasinoAtlanticCity > 2. Control > GET Obtener detalle carga

GET http://localhost:5287/api/cargas/8b570706-3f06-4e57-a94a-106a1157137e/contenido

Auth type: Bearer Token

Token: {{accessToken}}

200 OK • 472 ms • 2.21 KB

```

1  {
2    "idCarga": "8b570706-3f06-4e57-a94a-106a1157137e",
3    "registros": [
4      {
5        "id": "480da470-ab70-4109-ba6f-76509ffda1ba",
6        "idCarga": "8b570706-3f06-4e57-a94a-106a1157137e",
7        "periodo": "202601",
8        "codigoProducto": "PROD001",
9        "descripcion": "Producto 1",
10       "precio": 100.00,
11       "fechaRegistro": "2026-09-22T07:15:07.760022Z"
12     },
13     {
14       "id": "5dd6fcd5-c8aa-458f-b673-ed16ab6d671d",
15       "idCarga": "8b570706-3f06-4e57-a94a-106a1157137e",
16       "periodo": "202601",
17       "codigoProducto": "PROD002",
18       "descripcion": "Producto 2",
19       "precio": 150.00,

```

Reglas de negocio

Body Cookies Headers 4 Test Results 200 OK

JSON Preview Visualize

```

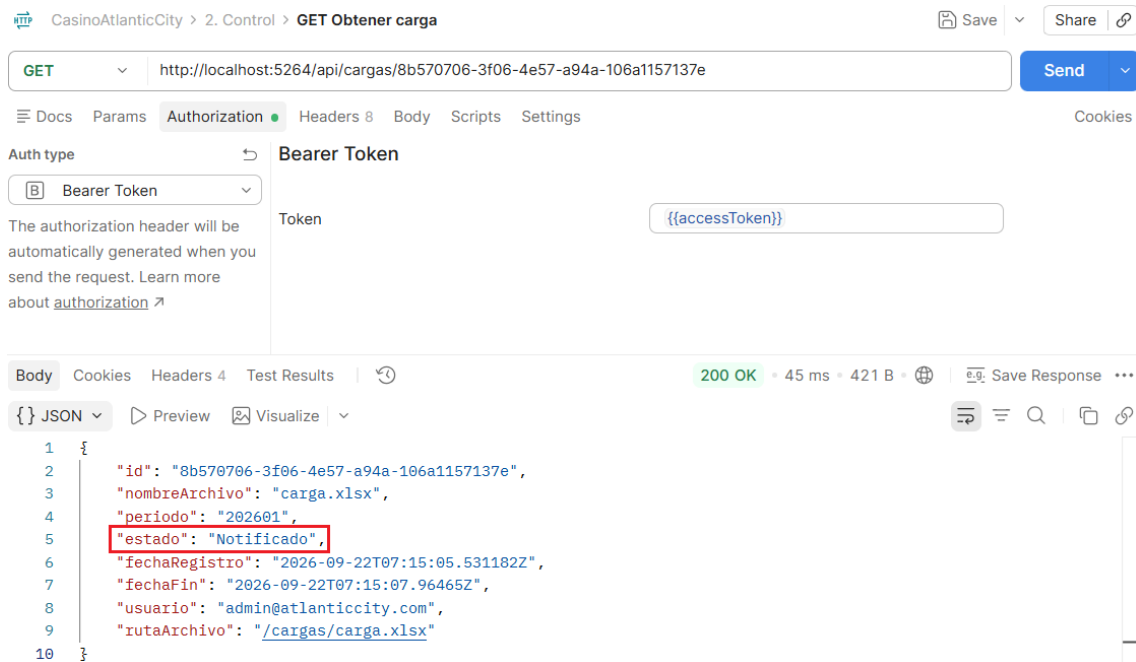
68  "errores": [
69    {
70      "id": "bf2487ff-00d9-4442-a140-e96acae27b55",
71      "idCarga": "8b570706-3f06-4e57-a94a-106a1157137e",
72      "fila": 5,
73      "codigoProducto": "PROD001",
74      "observacion": "Existente",
75      "fechaRegistro": "2026-09-22T07:15:07.821649Z"
76    },
77    {
78      "id": "7ae7eaa8-09ec-46fd-ba0e-656e87d7212c",
79      "idCarga": "8b570706-3f06-4e57-a94a-106a1157137e",
80      "fila": 7,
81      "codigoProducto": "",
82      "observacion": "CodigoProducto obligatorio",
83      "fechaRegistro": "2026-09-22T07:15:07.838927Z"
84    }
85  ]
86  }

```

4.4 Verificar procesamiento

Consultar el estado de la carga mediante:

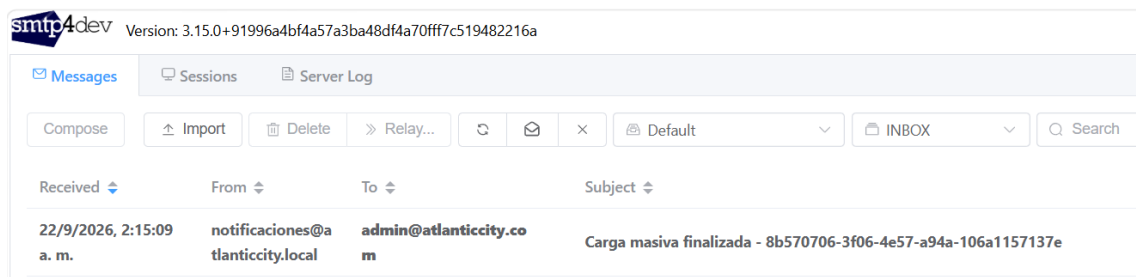
GET /cargas/{idCarga}



El procesamiento es asíncrono y pasa por RabbitMQ antes de ser procesado por el servicio de carga masiva.

4.5 Notificación

Una vez finalizado el procesamiento, verificar la notificación generada por el sistema.



5. Servicios utilizados

La solución utiliza los siguientes componentes:

- API Gateway
- Authentication
- Control
- CargaMasiva
- Notificaciones
- RabbitMQ
- PostgreSQL
- smtp4dev

6. Detener el proyecto

Para detener los contenedores:

docker compose down

Para volver a levantar el proyecto posteriormente:

docker compose up -d